# Probabilistic Machine Learning

## Lecture 05: Bayesian Networks

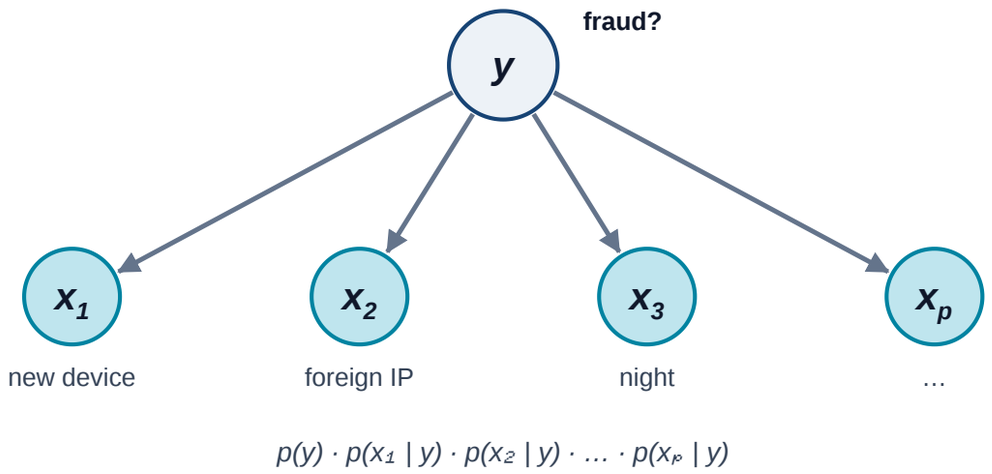
Prof. Dr. Michael Granitzer

Speaker notes

Lecture 05 is shortcut 4 of the course, "graph the joint". Naive Bayes (Lecture 03) made the joint affordable by one blanket conditional-independence assumption; a Bayesian network states only selected independences, and the graph encodes exactly those.

Contents: conditional independence checked by counting, the wet-grass model and its factorisation, explaining away (Factorising the Joint); belief networks, the local Markov property and parameter counts (Belief Networks); chains, forks, colliders, d-separation and Markov equivalence (d-Separation); plate notation, with the two models of Lecture 07 drawn (Plate Notation); variable elimination, Markov networks and moralisation, the intractability of exact inference on a pixel lattice, and parameter learning as counting (Inference and Parameter Learning). The appendix covers soft and unreliable evidence.

# Where We Left Off




Lecture 04 skipped $p(x)$. Lecture 03 kept the whole joint and made it affordable with one blanket assumption: **every** attribute independent of every other, given the class. It worked — and it counted correlated evidence twice.

**What if we assumed only the independences we actually believe?**

**Shortcut 4 — graph the joint.** Draw which variables depend on which; the graph *is* the factorisation. Dropped: every dependence **not drawn** — and, for a dense graph, affordable inference.

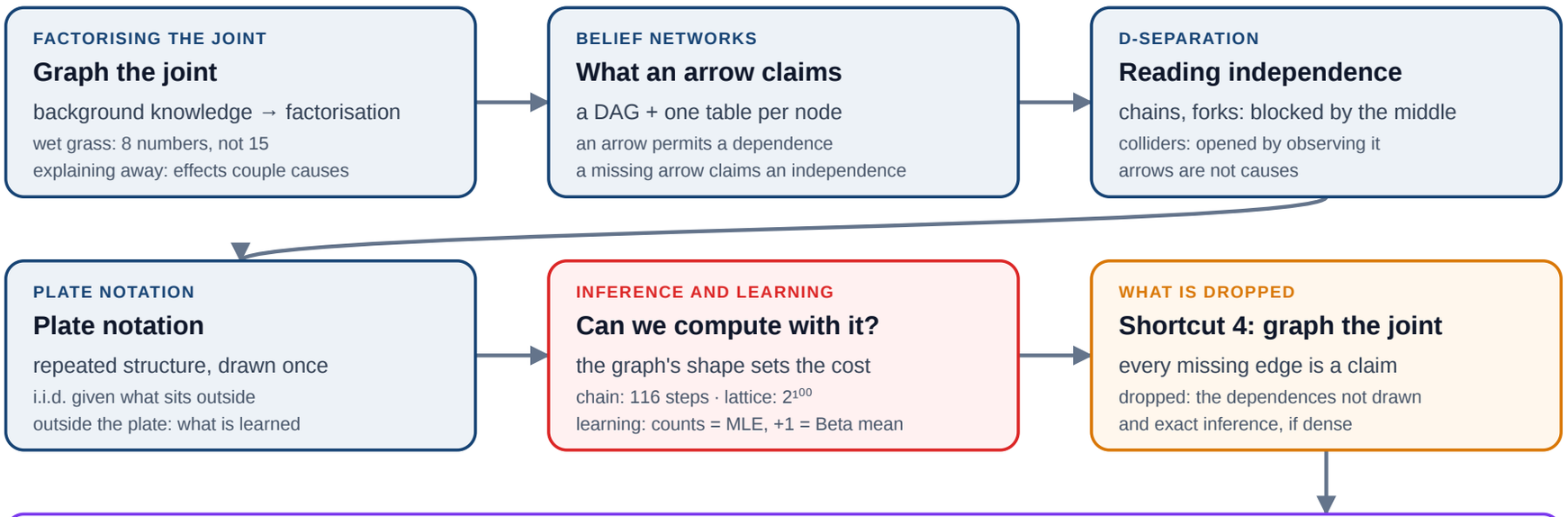
Speaker notes

A discriminative model (Lecture 04) fits only $p(y \mid x)$ and drops $p(x)$. Naive Bayes keeps the whole joint $p(y, x_1, \ldots, x_d)$ and makes it affordable by assuming all attributes conditionally independent given the class. Drawn as a graph, this is the star on the left: the class points to every attribute and no arrow joins two attributes. Its known failure is double counting: two strongly correlated attributes enter the posterior as two independent pieces of evidence, which pushes the posterior too far.

"Graph the joint" generalises the star: any directed acyclic graph defines a factorisation, and the modeller chooses which independences to assume. Naive Bayes is the special case with the star graph. Shortcut 4 has two costs: every missing arrow is an independence claim that may be false (Factorising the Joint, Belief Networks, d-Separation), and a graph that is compact to write down may still be too expensive to condition on exactly (Inference and Parameter Learning).

# Argument Map

**Lecture 05 · argument map**




**Course arc**
ideal → shortcuts

| L01 | L02 | L03 | L04 | L05 | L06 | L07 |
|---|---|---|---|---|---|---|
| The ideal | The language | Apart · peak | Only p(y \| x) | **Graph the joint** | Keep the loss | No shortcut |

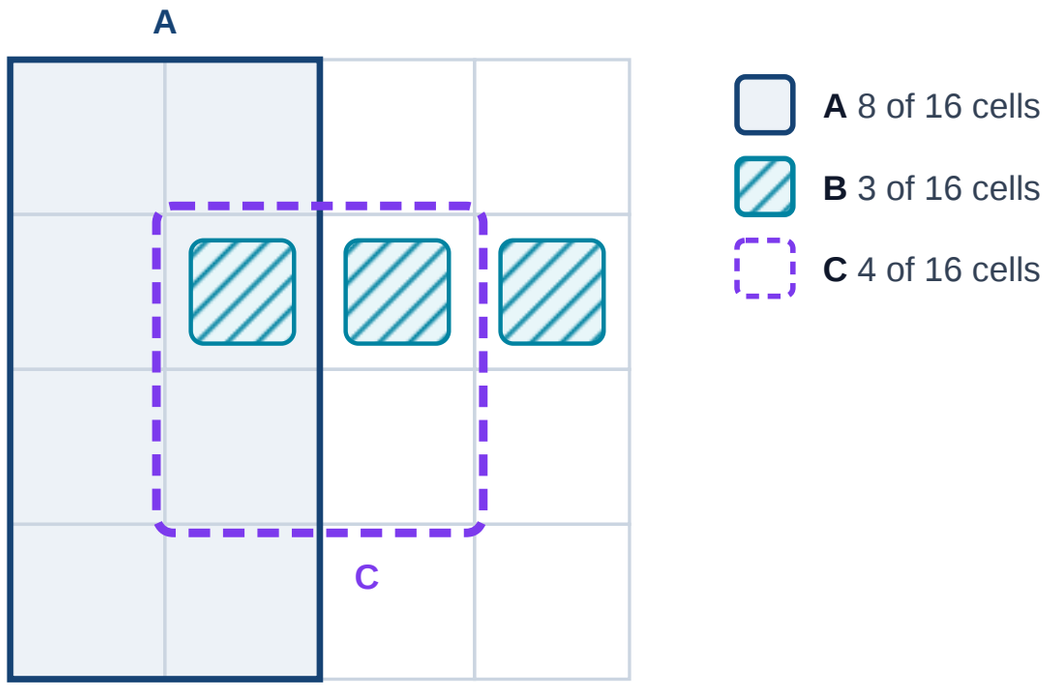
Speaker notes

Top row: the joint written as a graph, the claims its arrows and missing arrows make, and how conditional independences are read off the graph. Middle row: plate notation for repeated structure, the cost of querying a graph, and shortcut 4 with what it drops — every dependence not drawn and, for densely connected graphs, tractable inference. The single conclusion: a model that is cheap to write down (few parameters) can still be intractable to condition on, because the cost of exact inference depends on the graph's shape, not on its number of parameters.

# Factorising the Joint

**Guiding Question:** Naive Bayes assumed every attribute independent of every other. What if we stated only the independences we believe — and what does conditioning then do to them?

# Independent Given C — Counted




Sixteen equally likely cells.

## Dependent

$$P(A \cap B) = \frac{1}{16} \neq \frac{3}{32} = P(A)P(B)$$

## Independent given $C$

$$P(A \cap B \mid C) = \frac{1}{4} = P(A \mid C)P(B \mid C)$$

Speaker notes

$A$ and $B$ are conditionally independent given $C$ if $P(A \cap B \mid C) = P(A \mid C)\,P(B \mid C)$, equivalently $P(A \mid B \cap C) = P(A \mid C)$ whenever $P(B \cap C) > 0$. On the grid: $P(A) = 8/16$, $P(B) = 3/16$, $P(A \cap B) = 1/16$, so $A$ and $B$ are dependent ($1/16 \neq 3/32$). Inside the block $C$ (4 cells), 2 cells are in $A$, 2 in $B$ and 1 in both: $1/4 = 1/2 \cdot 1/2$. Also $P(A \mid B \cap C) = 1/2 = P(A \mid C)$.

Neither property implies the other: marginal independence does not imply conditional independence, and conditional independence does not imply marginal independence. A typical source of the second case is a common cause: ice-cream sales and drownings are dependent, but approximately independent given the temperature. Conditioning on a common cause removes a dependence; conditioning on a common effect can create one (explaining away, later in this module).

# Why Is the Grass Wet?

Tracey sees her lawn is wet. **Rain — or did she leave the sprinkler on?** Then she sees that her neighbour Jack's lawn is wet too.

| variable | meaning (1 = yes) |
|---|---|
| $R$ | it rained |
| $S$ | Tracey's sprinkler was on |
| $J$ | Jack's grass is wet |
| $T$ | Tracey's grass is wet |

**Before any arithmetic: does Jack's wet lawn make the sprinkler more or less likely?**

Speaker notes

The example is from Barber, *Bayesian Reasoning and Machine Learning*, Chapter 3. The answer is computed two slides on: Tracey's wet lawn alone raises $P(S{=}1)$ from 0.1 to 0.338; learning that Jack's lawn is also wet lowers it to 0.160. Jack's lawn is evidence for rain, rain explains Tracey's wet lawn, so the sprinkler is needed less as an explanation.

A common intuition is that more evidence of wetness should make every wet cause more likely. It is wrong here because Jack's lawn depends on rain but not on the sprinkler: the extra evidence favours one cause specifically, and the two causes compete for the same effect.

# Background Knowledge as Independence

The joint needs $2^4 - 1 = 15$ numbers. The chain rule rewrites it without shrinking it:

$$p(T, J, R, S) = p(T \mid J, R, S) \cdot p(J \mid R, S) \cdot p(R \mid S) \cdot p(S)$$

- → Tracey's lawn ignores Jack's, given $R$ and $S$: $p(T \mid J, R, S) = p(T \mid R, S)$
- → Jack's lawn ignores Tracey's sprinkler: $p(J \mid R, S) = p(J \mid R)$
- → the sprinkler does not change the weather: $p(R \mid S) = p(R)$

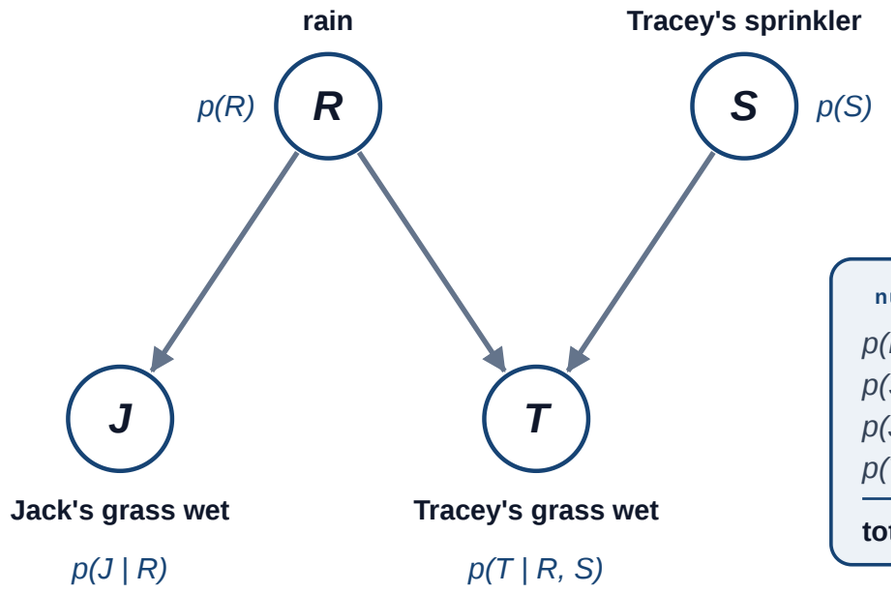
> $$p(T, J, R, S) = p(T \mid R, S) \cdot p(J \mid R) \cdot p(R) \cdot p(S)$$

Speaker notes

The chain rule holds for any ordering of the variables and never reduces the number of parameters: with four binary variables the factors need $8 + 4 + 2 + 1 = 15$ numbers, the same as the joint. The saving comes only from the three conditional-independence assumptions, each of which is domain knowledge and can fail. If Tracey and Jack share a garden hose, Jack's lawn depends on the sprinkler and $p(J \mid R, S) \neq p(J \mid R)$. If the sprinkler is switched on by a rain sensor, $p(R \mid S) \neq p(R)$.

After the assumptions the factors need $4 + 2 + 1 + 1 = 8$ numbers. Naive Bayes makes one blanket assumption (all attributes independent given the class); here three specific claims are made and all other dependences are kept. The ordering matters for how compact the result is: choosing an order in which causes precede effects usually gives the fewest parents.

# The Graph Is the Factorisation




| numbers needed | |
|---|---|
| *p(R)* | **1** |
| *p(S)* | **1** |
| *p(J \| R)* | **2** |
| *p(T \| R, S)* | **4** |
| **total** | **8** |

full joint: 15

| factor | $= 1$ |
|---|---|
| $p(R = 1)$ | 0.2 |
| $p(S = 1)$ | 0.1 |
| $p(J = 1 \mid R = 0)$ | 0.2 |
| $p(J = 1 \mid R = 1)$ | 1.0 |
| $p(T = 1 \mid R = 0, S = 0)$ | 0.0 |
| $p(T = 1 \mid R = 0, S = 1)$ | 0.9 |
| $p(T = 1 \mid R = 1, S = 0)$ | 1.0 |
| $p(T = 1 \mid R = 1, S = 1)$ | 1.0 |

One node per variable, one arrow per conditioning bar, one table per node.

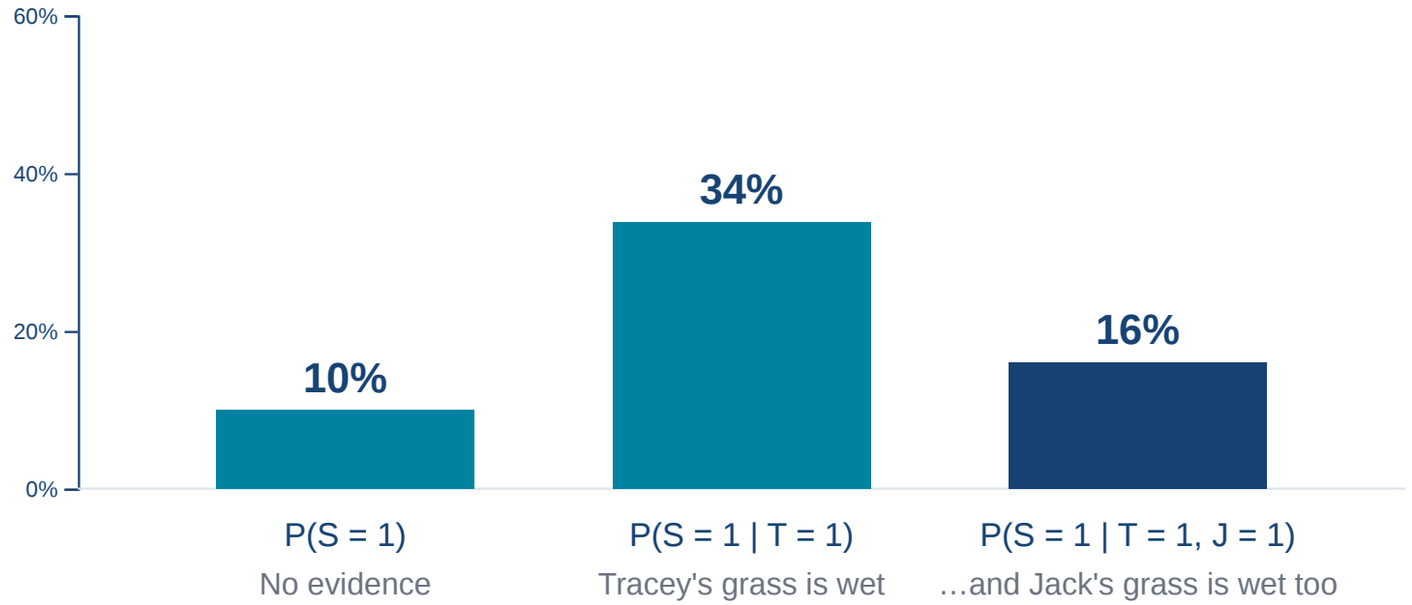
Speaker notes

Each factor $p(\text{node} \mid \text{parents})$ corresponds to one node and its incoming arrows: $R$ and $S$ have no parents, $J$ has parent $R$, $T$ has parents $R$ and $S$. For binary variables each node needs one number per parent configuration: $1 + 1 + 2 + 4 = 8$, the eight rows of the table.

Some entries are modelling choices rather than estimates: $p(T{=}1 \mid R{=}0, S{=}0) = 0$ says the grass never gets wet without rain or sprinkler, and $p(J{=}1 \mid R{=}1) = 1$ says rain always wets Jack's lawn. Probabilities of exactly 0 or 1 cannot be changed by any amount of evidence under Bayes' rule, so they are strong claims. Inference and Parameter Learning shows that such values arise naturally as maximum-likelihood estimates from small samples.

# What Does the Evidence Say?

Rain explains the wetness — the sprinkler falls back towards its prior



Speaker notes

Step 1 conditions on $T=1$: $P(T{=}1) = 0.2 + 0.8 \cdot 0.1 \cdot 0.9 = 0.272$ and $P(S{=}1, T{=}1) = 0.02 + 0.072 = 0.092$, so $P(S{=}1 \mid T{=}1) = 0.338$, up from the prior 0.1. Step 2 adds $J=1$: $P(S{=}1 \mid T{=}1, J{=}1) = 0.0344 / 0.2144 = 0.160$.

The sprinkler probability falls most of the way back but stays above its prior: rain explains the wet lawn completely in this model, yet the sprinkler may still have been on in addition, and the evidence does not rule rain out ($p(J{=}1 \mid R{=}0) = 0.2$). Both values are obtained by exact enumeration of the factorised joint and marginalisation; no new rule of probability is needed.

# Explaining Away

$$p(S=1 \mid T=1, J=1) = \frac{\sum_R p(T{=}1 \mid R, S{=}1)p(J{=}1 \mid R)p(R)p(S{=}1)}{\sum_{R,S} p(T{=}1 \mid R, S)p(J{=}1 \mid R)p(R)p(S)} = \frac{0.0344}{0.2144} \approx 0.16$$

Rain and sprinkler are independent: $R \perp\!\!\!\perp S$. Once their **common effect** $T$ is observed they compete to explain it, and evidence for one lowers the other. Conditioning on a common cause removes a dependence; conditioning on a common effect **creates** one.

Speaker notes

Numerator, $S = 1$: with rain $0.2 \cdot 0.1 \cdot 1 \cdot 1 = 0.02$; without rain $0.8 \cdot 0.1 \cdot 0.9 \cdot 0.2 = 0.0144$; sum 0.0344. Denominator adds $S = 0$: with rain $0.2 \cdot 0.9 \cdot 1 \cdot 1 = 0.18$; without rain 0, because $p(T{=}1 \mid R{=}0, S{=}0) = 0$. Total $0.2144$.

$R \perp\!\!\!\perp S$ holds marginally because neither has a parent and no path between them avoids $T$. Conditioning on $T$ (or on any descendant of $T$) makes them dependent. The 16-cell example and the wet grass are the two directions: a common cause induces dependence that conditioning removes; a common effect leaves its causes independent until conditioning creates a dependence. The d-separation module turns both into a rule that applies to any graph.

# Mini-Exercise

Tracey's lawn is wet. Then she finds the **sprinkler was on**.

**Does rain become more or less likely than it was a moment ago?**

Speaker notes

$P(R{=}1, T{=}1) = 0.2$ and $P(T{=}1) = 0.272$, so $P(R{=}1 \mid T{=}1) = 0.735$. With the sprinkler known to be on: $P(R{=}1, S{=}1, T{=}1) = 0.02$ and $P(S{=}1, T{=}1) = 0.092$, so $P(R{=}1 \mid T{=}1, S{=}1) = 0.217$, close to the prior 0.2.

It does not return exactly to 0.2 because $T{=}1$ still carries a little information about rain when $S{=}1$: $p(T{=}1 \mid R{=}1, S{=}1) = 1$ but $p(T{=}1 \mid R{=}0, S{=}1) = 0.9$, so a wet lawn is slightly more likely with rain. Explaining away is symmetric: evidence for either cause of a common effect lowers the probability of the other.

# Mini-Exercise

Tracey's lawn is wet. Then she finds the **sprinkler was on**.

## Does rain become more or less likely than it was a moment ago?

**Less.** The sprinkler explains the wet lawn. $P(R = 1 \mid T = 1) \approx 0.74$ falls to $P(R = 1 \mid T = 1, S = 1) = 0.02/0.092 \approx 0.22$ — almost back to the prior 0.2. Explaining away works in both directions.

# Takeaway

**Graph the joint:** background knowledge becomes conditional independence, and the graph is a picture of how the joint splits — 8 numbers instead of 15. Every *missing* edge is a claim; conditioning can make variables independent (common cause) or dependent (common effect).

# Belief Networks

**Guiding Question:** What exactly does an arrow in a probabilistic model claim — and what does a *missing* arrow claim?

# Belief Networks

**Bayesian belief network.** A **directed acyclic graph** whose nodes are random variables, with one conditional table per node given its parents:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p\big(x_i \mid \mathrm{pa}(x_i)\big)$$

**An arrow permits a dependence. A missing arrow claims an independence:** each variable is independent of its non-descendants, given its parents (the **local Markov property**).

**Which independence does the wet-grass graph claim for Jack's lawn $J$?**

Speaker notes

The product is a valid distribution for any choice of conditional tables: order the nodes so that parents precede children (always possible in a DAG); summing out the variables from the last to the first then removes one normalised factor at a time. With a cycle no such order exists, and a product of conditionals need not define a joint distribution.

The non-descendants of $J$ are $R$, $S$ and $T$ ($T$ is not reachable from $J$), so the local Markov property gives $J \perp\!\!\!\perp \{S, T\} \mid R$.

An arrow only permits a dependence: a table may ignore a parent. In the wet-grass tables $p(T{=}1 \mid R{=}1, S) = 1$ for both values of $S$, so given rain the arrow $S \to T$ has no effect. A missing arrow, by contrast, enforces an independence for every choice of tables on that graph. The claims a graph makes are therefore carried by its missing edges; a fully connected DAG claims no independence and can represent every distribution.

# Belief Networks

**Bayesian belief network.** A **directed acyclic graph** whose nodes are random variables, with one conditional table per node given its parents:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p\big(x_i \mid \mathrm{pa}(x_i)\big)$$

**An arrow permits a dependence. A missing arrow claims an independence:** each variable is independent of its non-descendants, given its parents (the **local Markov property**).

## Which independence does the wet-grass graph claim for Jack's lawn $J$?

$J \perp\!\!\!\perp \{S, T\} \mid R$: once we know whether it rained, nothing else says more about Jack's lawn.

# Counting the Numbers

| model, binary variables | full joint | graph |
|---|---|---|
| wet grass, 4 nodes | 15 | $1 + 1 + 2 + 4 = 8$ |
| naive Bayes, class + 30 flags | $2^{31} - 1 \approx 2.1 \cdot 10^9$ | $1 + 2 \cdot 30 = 61$ |
| 20 nodes, at most 3 parents each | $2^{20} - 1 \approx 10^6$ | at most $20 \cdot 2^3 = 160$ |

**Where does the saving come from — and what makes it disappear?**

Speaker notes

A node with $k$ binary parents needs $2^k$ free numbers ($2^k$ parent configurations, one Bernoulli parameter each); a root node needs one. Wet grass: $R$ and $S$ are roots, $J$ has one parent, $T$ two, so $1 + 1 + 2 + 4 = 8$.

Naive Bayes with a binary class and 30 binary flags: one class prior and, for every flag, one Bernoulli parameter per class, $1 + 2 \cdot 30 = 61$ (Lecture 03). The full joint over 31 binary variables needs $2^{31} - 1 = 2{,}147{,}483{,}647$ numbers.

With 20 nodes and at most 3 parents each, every table has at most 8 numbers: at most 160, against $2^{20} - 1 = 1{,}048{,}575$. In general, $n$ binary nodes with at most $k$ parents need at most $n \cdot 2^k$ numbers — linear in $n$, while the joint grows as $2^n$. The bound is exponential in $k$, so a single node with many parents (a symptom with 30 possible causes needs $2^{30}$ rows) removes the saving.

# Counting the Numbers

| model, binary variables | full joint | graph |
|---|---|---|
| wet grass, 4 nodes | 15 | $1 + 1 + 2 + 4 = 8$ |
| naive Bayes, class + 30 flags | $2^{31} - 1 \approx 2.1 \cdot 10^9$ | $1 + 2 \cdot 30 = 61$ |
| 20 nodes, at most 3 parents each | $2^{20} - 1 \approx 10^6$ | at most $20 \cdot 2^3 = 160$ |

## Where does the saving come from — and what makes it disappear?

A node with $k$ binary parents needs $2^k$ numbers. The saving comes from **missing arrows**; it disappears as soon as some node has many parents.

# Keeping Tables Small: Noisy-OR

A symptom $y$ with many possible causes $x_1, \ldots, x_k$: **each present cause fires on its own** with probability $\lambda_i$, and $y$ occurs if any fires:

$$p(y = 0 \mid x_1, \ldots, x_k) = \prod_{i : x_i = 1} (1 - \lambda_i)$$

$k$ numbers instead of $2^k$. Two causes present, $\lambda = 0.8$ and $0.6$: $p(y = 1) = 1 - 0.2 \cdot 0.4 = 0.92$.

**THE ASSUMPTION BOUGHT**

Causes act **independently** — no interaction. Two drugs that are harmless alone but dangerous together break it.

**Divorcing** is the other trick: insert a summary node ("total load") between a group of parents and the child.

Speaker notes

Derivation: each present cause $i$ independently fails to trigger $y$ with probability $1 - \lambda_i$, and $y = 0$ only if all present causes fail, which gives the product. With $\lambda = 0.8$ and $0.6$: $p(y = 0) = 0.2 \cdot 0.4 = 0.08$, so $p(y = 1) = 0.92$. Adding a present cause never lowers $p(y = 1)$.

As written, noisy-OR has no leak: with no cause present, $p(y = 1) = 0$. A leak term $\lambda_0$ covers causes outside the model: $p(y = 0 \mid x) = (1 - \lambda_0) \prod_{i:x_i=1} (1 - \lambda_i)$.

The independence assumption fails whenever causes interact, i.e. their joint effect differs from independent triggering: drug interactions, or two conditions that produce a symptom only together (an AND rather than an OR).

Divorcing inserts an intermediate variable that summarises a subset of the parents. A child with six binary parents needs 64 numbers; splitting them into two groups of three, each summarised by a binary node, needs $8 + 8 + 4 = 20$. The assumption is that the child depends on each group only through its summary. Smaller tables are also easier to elicit from experts.

# Mini-Exercise

Tracey and Jack share a garden hose: add the arrow $S \to J$ to the wet-grass network.

**How many numbers does the network need now — and which independence claim is gone?**

Speaker notes

Only $J$'s table changes: with parents $R$ and $S$ it has 4 rows instead of 2, so the total is $1 + 1 + 4 + 4 = 10$. The removed claim is $J \perp\!\!\!\perp S \mid R$. Marginal independence of $J$ and $S$ disappears too: without the arrow the only path $J \leftarrow R \rightarrow T \leftarrow S$ is blocked by the unobserved collider $T$; with it, $J$ and $S$ are dependent even when nothing is observed.

In general, adding an arrow removes independence claims and costs parameters; removing one adds a claim and saves parameters. The graph must stay acyclic, so not every arrow can be added.

# Mini-Exercise

Tracey and Jack share a garden hose: add the arrow $S \to J$ to the wet-grass network.

**How many numbers does the network need now — and which independence claim is gone?**

**10.** $p(J \mid R, S)$ needs 4 numbers instead of 2: $1 + 1 + 4 + 4$. Gone is $J \perp\!\!\!\perp S \mid R$ — the missing arrow was that claim.

# Takeaway

A belief network is **a DAG plus one table per node**. An arrow only permits a dependence; every **missing** arrow claims an independence, and each claim saves numbers. Shrinking tables further — noisy-OR, divorcing — buys more savings with more assumptions, which should be said out loud.

# d-Separation

**Guiding Question:** Two causes are independent until you observe their shared effect. Can we see such things in the picture — before computing a single number?

# Three Ways Three Nodes Connect

### CHAIN

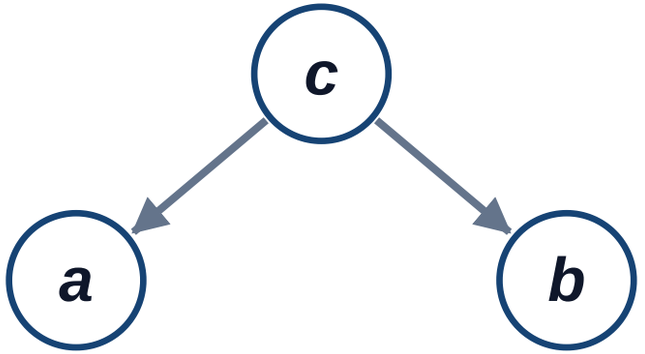



a, b: **dependent**

given c: **independent**

**observing c blocks the path**

### FORK · COMMON CAUSE

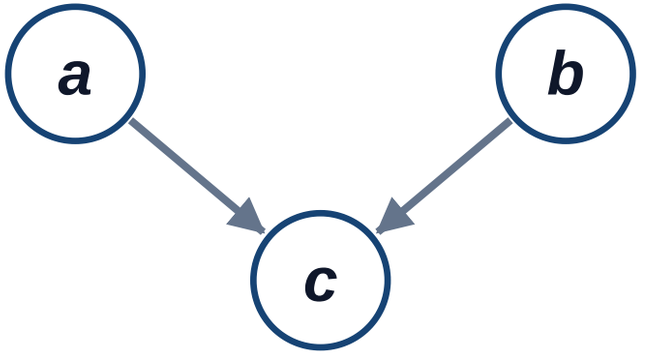



a, b: **dependent**

given c: **independent**

**observing c blocks the path**

### COLLIDER · COMMON EFFECT




a, b: **independent**

given c: **dependent**

**observing c opens the path**

Every independence question in a DAG reduces to these three.

Speaker notes

Chain $a \to c \to b$: $p(a,b,c) = p(a)\,p(c \mid a)\,p(b \mid c)$, so $a \perp\!\!\!\perp b \mid c$; marginally $a$ and $b$ are in general dependent. Fork $a \leftarrow c \to b$: $p(c)\,p(a \mid c)\,p(b \mid c)$, again $a \perp\!\!\!\perp b \mid c$ and in general dependent marginally. The fork is the common cause (temperature for ice-cream sales and drownings).

Collider $a \to c \leftarrow b$: $p(a)\,p(b)\,p(c \mid a,b)$. Summing out $c$ gives $p(a)\,p(b)$, so $a \perp\!\!\!\perp b$ marginally; given $c$ the factor $p(c \mid a,b)$ couples them, and in general $a \not\perp\!\!\!\perp b \mid c$. This is the common effect (rain → wet lawn ← sprinkler) and the only pattern in which observing the middle node opens the path instead of blocking it.

# d-Separation

A path between $X$ and $Y$ is **blocked** by a set $Z$ if it contains

- a chain or fork whose middle node **is in** $Z$, or
- a collider whose middle node — and every descendant of it — is **not in** $Z$.

If every path is blocked, $X$ and $Y$ are **d-separated** by $Z$, and then $X \perp\!\!\!\perp Y \mid Z$ in every distribution that factorises over the graph.

Sound, but not complete for a particular table: specific numbers can create independences the graph does not show. For almost every table that factorises over the graph, d-separation finds them all.

Speaker notes

The descendant clause: if a descendant of the collider is observed, it is indirect evidence about the collider, so the two causes again compete to explain it and the path is open.

d-separation is sound: if $Z$ d-separates $X$ and $Y$, then $X \perp\!\!\!\perp Y \mid Z$ in every distribution that factorises over the graph. It is also complete with respect to the graph: if $X$ and $Y$ are not d-separated, there is some distribution on the graph in which they are dependent (indeed, almost every choice of tables makes them dependent). What d-separation does not capture are additional independences produced by particular numbers in the tables.

The wet-grass example on the slide is of this kind, but weaker than a full conditional independence: $T \perp\!\!\!\perp S \mid R{=}1$ holds (rain wets the lawn regardless of the sprinkler), while $T \perp\!\!\!\perp S \mid R{=}0$ does not. Such a context-specific independence is a property of the tables, not of the graph. Distributions without any extra independences are called faithful to the graph.

# d-Separation on the Wet Grass

The only path from $J$ to $S$: $J \leftarrow R \rightarrow T \leftarrow S$ — a fork at $R$, a collider at $T$.

| query | path | verdict | the numbers |
|---|---|---|---|
| $J \perp\!\!\!\perp S$ ? | collider $T$ unobserved: blocked | independent | $P(J{=}1) = 0.36 = P(J{=}1 \mid S{=}1)$ |
| $J \perp\!\!\!\perp S \mid T$ ? | collider $T$ observed: open | dependent | $P(J{=}1 \mid T{=}1) = 0.79$, but $0.37$ if also $S{=}1$ |
| $J \perp\!\!\!\perp S \mid T, R$ ? | fork $R$ observed: blocked | independent | $P(S{=}1 \mid T{=}1, R{=}1) = 0.1$, with or without $J{=}1$ |

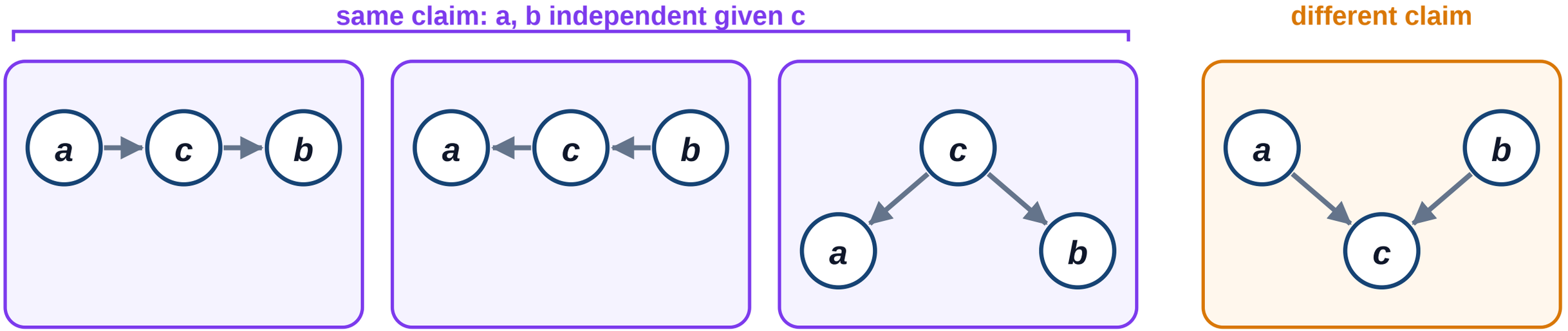
> The middle row is the explaining away of Factorising the Joint: $10\% \rightarrow 34\% \rightarrow 16\%$ for the sprinkler. The graph predicted it before any number was computed.

Speaker notes

Exact values by enumeration of the factorised joint: $P(J{=}1) = 0.2 \cdot 1 + 0.8 \cdot 0.2 = 0.36$, and the same given $S{=}1$ because $J$ depends on $S$ only through the blocked collider. $P(J{=}1 \mid T{=}1) = 0.2144 / 0.272 = 0.788$; with $S{=}1$ in addition, $P(J{=}1 \mid T{=}1, S{=}1) = 0.0344 / 0.092 = 0.374$. The sprinkler explains Tracey's lawn, rain becomes less likely, and with it Jack's wet lawn. Given rain, $T{=}1$ is certain whatever $S$ is, so $P(S{=}1 \mid T{=}1, R{=}1)$ equals the prior 0.1, with or without $J{=}1$.

A path is checked triple by triple: it is blocked if any one triple on it is blocked. Here the fork at $R$ blocks when $R$ is observed, the collider at $T$ blocks unless $T$ or a descendant is observed. d-separation thus decides from the graph alone whether a piece of evidence can affect a query; inference algorithms use this to discard irrelevant parts of a network.

# Arrows Are Not Causes




data cannot tell these three apart

the direction of the arrows is not in the data

a, b independent;
dependent given c

Same skeleton, same colliders: **the same independences** (Markov equivalence). Data cannot orient such arrows — they are **not** causal claims.

Speaker notes

Two DAGs are Markov equivalent if they imply the same set of conditional independences. Verma and Pearl (1990) showed this holds exactly when they have the same skeleton (edges ignoring direction) and the same v-structures (colliders $a \to c \leftarrow b$ whose ends $a$, $b$ are not adjacent). The chain in either direction and the fork all claim only $a \perp\!\!\!\perp b \mid c$; the collider claims $a \perp\!\!\!\perp b$ instead.

The wet-grass arrows point in the causal direction because the causal story is known, not because the independences require it. Only v-structures leave a signature in observational data (two independent variables that become dependent given a third), so structure-learning algorithms can orient some edges and leave others undetermined. A causal claim needs information beyond the observed distribution: interventions, temporal order, or explicitly stated assumptions. Inference and Parameter Learning states the same limit for structure learning.

# Mini-Exercise

Tracey's doormat gets wet whenever her lawn is: add $T \to D$. You see only a **wet doormat —** not the lawn.

**Are rain and sprinkler still independent?**

Speaker notes

$D$ is a child of the collider $T$, so observing $D$ is indirect evidence about $T$: a wet doormat raises the probability of a wet lawn, and the two causes of the lawn compete again. The path $R \to T \leftarrow S$ is open given $D$, so $R \not\perp\!\!\!\perp S \mid D$. If the doormat is wet exactly when the lawn is ($D = T$), observing $D$ is the same as observing $T$; if the doormat can also get wet otherwise, the effect is weaker but still present.

With $D = T$: $P(S{=}1 \mid D{=}1) = 0.338$, and once rain is also observed, $P(S{=}1 \mid D{=}1, R{=}1) = 0.1$ — the sprinkler becomes less likely, exactly as on the lawn. The descendant clause of d-separation is the one most often forgotten.

# Mini-Exercise

Tracey's doormat gets wet whenever her lawn is: add $T \to D$. You see only a **wet doormat —** not the lawn.

## Are rain and sprinkler still independent?

**No.** $D$ is a descendant of the collider $T$, so observing it opens the path $R \to T \leftarrow S$: a wet doormat also makes rain and sprinkler compete to explain it.

# Takeaway

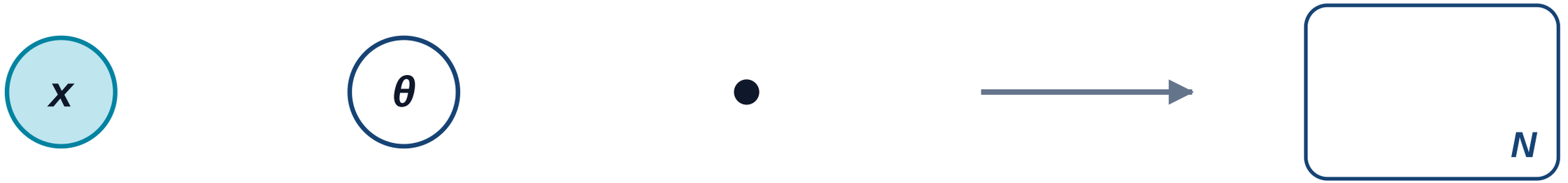
Independence in a belief network is a **property of the graph**: chains and forks are blocked by observing the middle, colliders are opened by it (or by a descendant). d-separation applies that to every path. What it cannot read off is the direction of cause: equivalent graphs make the same claims.

# Plate Notation

**Guiding Question:** A model of 10,000 observations should not need 10,000 nodes. How is a repeated structure drawn — and what does the drawing claim about the observations?

# Drawing a Model

A probabilistic model is a factorisation, and the diagram is its picture. Five conventions carry almost all of it:




**shaded**
observed: in the data

**open**
latent or parameter

**solid dot**
fixed, not inferred

**arrow**
child conditioned on parent

**plate**
repeat N times

> Nothing new is claimed: the arrows are read exactly as in a belief network. The shading only says **what the data fixes** — which variables we condition on.

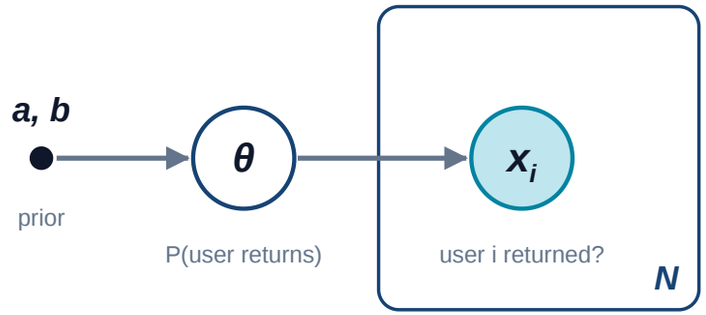
Speaker notes

These conventions (directed graphical models with plates, after Buntine 1994; standard in Bishop, *Pattern Recognition and Machine Learning*, Chapter 8) add no new semantics: arrows are read exactly as in a belief network, and the diagram encodes the factorisation $\prod_i p(x_i \mid \mathrm{pa}(x_i))$.

Shaded nodes are observed: the joint is conditioned on their values. Open nodes are latent variables or parameters with a distribution, to be inferred. A solid dot is a fixed quantity (a hyperparameter or a given input) that has no distribution in the model and therefore no factor of its own. Of the course's three moves — model the joint, condition on the data, read off answers — the second corresponds in the diagram to shading the observed nodes. Conventions differ between authors: some draw hyperparameters as small open circles or omit them.

# Plates

A **plate** is a box with a count in its corner: *repeat what is inside $N$ times*. A model of 10,000 users does not need 10,000 nodes.
The copies are **independent given what sits outside** — the parameters, which learning estimates. Nested plates draw grouped data: $K$ hospitals with $N_k$ patients each.



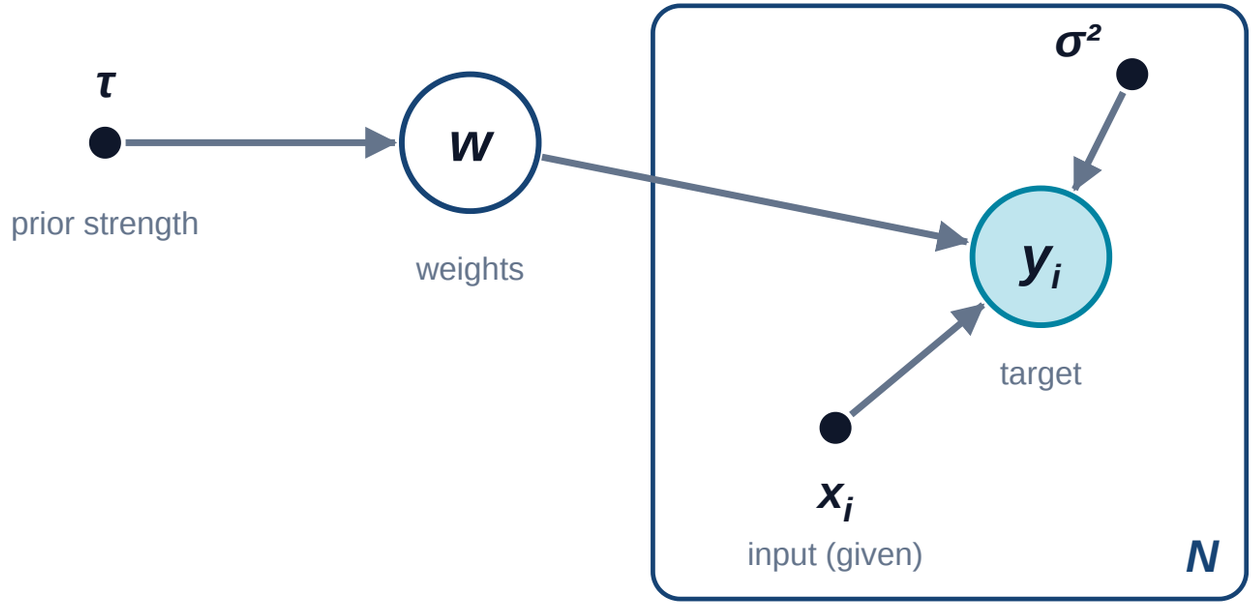

> Given $\theta$ the users are independent. With $\theta$ unknown they are **not**: in Lecture 03, the first ten users changed our prediction for the eleventh. That dependence is exactly why learning from data works.

Speaker notes

A plate stands for $N$ copies of its contents with the same parameters; the factorisation gets a product $\prod_{i=1}^{N}$ over the factors inside. The i.i.d. assumption of Lecture 03 is conditional: the $x_i$ are independent given $\theta$. Marginally, with $\theta$ summed out, they are dependent — the Beta–Bernoulli predictive is $P(x_{N+1} = 1 \mid x_{1:N}) = (a + k)/(a + b + N)$ with $k$ the number of ones so far, which depends on the earlier observations. In d-separation terms, every pair $x_i \leftarrow \theta \rightarrow x_j$ is a fork that is open while $\theta$ is unobserved.

A frequent error is to read "i.i.d." as "the data says nothing about the next point"; that holds only given the parameters. Variables shared across copies must sit outside the plate; a variable inside a plate has one copy per index.

# Reading a Diagram




Read off the factorisation:

$$p(w \mid \tau) \prod_{i=1}^{N} p(y_i \mid x_i, w, \sigma^2)$$

The plate became the product; each arrow became a conditioning bar.

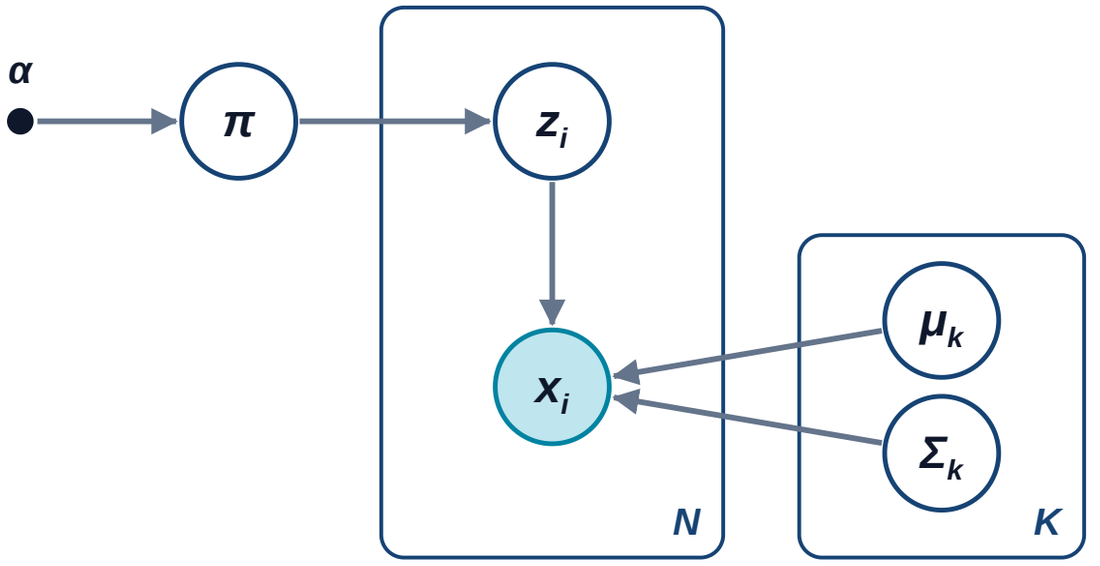
$x_i$ is a **dot**: the inputs are given, not modelled — this is a discriminative model, $p(y \mid x)$, with a prior on $w$. Lecture 07 conditions on the shaded $y_i$ and keeps the whole posterior over $w$.

Speaker notes

This is the model of Lecture 07: a prior $p(w \mid \tau)$ and Gaussian likelihood $p(y_i \mid x_i, w, \sigma^2) = \mathcal{N}(y_i \mid w^\top x_i, \sigma^2)$. The inputs $x_i$ are dots: they are conditioned on but not modelled, so the factorisation has no factor $p(x_i)$. This is the discriminative choice of Lecture 04, visible directly in the notation; a generative model would draw $x_i$ as a shaded random node with its own distribution.

The noise variance $\sigma^2$ is also a dot, i.e. fixed. If it were an open node it would need a prior, and the posterior would be over $(w, \sigma^2)$ jointly; with a Gaussian prior on $w$ and an inverse-gamma prior on $\sigma^2$ that posterior is still available in closed form (normal–inverse-gamma).

# Mini-Exercise



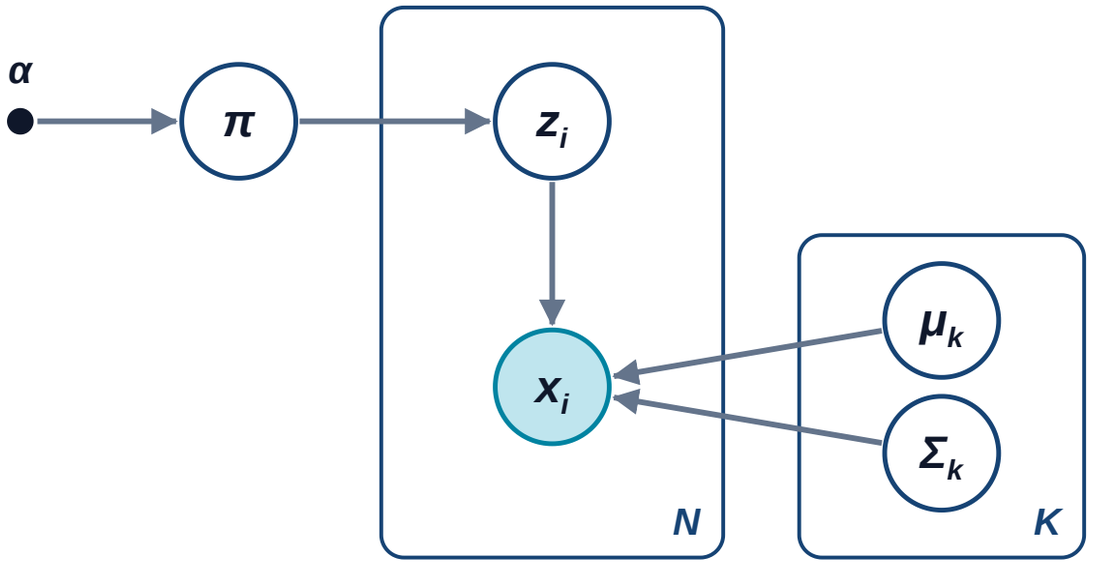

**Which nodes does learning have to infer — and why is $\alpha$ not among them?**

Speaker notes

Every open node without a parent gets a prior factor ($\pi$ via $\alpha$; $\mu_k, \Sigma_k$ directly); every arrow is a conditioning bar. $\alpha$ is a dot, a fixed hyperparameter, so it is set by the modeller, not inferred. The $K$ plate holds the component parameters, one copy per component; the $N$ plate holds $z_i$ and $x_i$, one copy per data point.

$x_i$ has arrows from $z_i$ and from every $(\mu_k, \Sigma_k)$: $p(x_i \mid z_i, \mu, \Sigma) = \mathcal{N}(x_i \mid \mu_{z_i}, \Sigma_{z_i})$, so $z_i$ selects the component. The latent variables split into two kinds: global ($\pi$, $\mu_k$, $\Sigma_k$; their number does not grow with $N$) and local ($z_i$; one per data point). EM in Lecture 07 computes a distribution over the local $z_i$ and point estimates of the global parameters.

# Mini-Exercise




**Which nodes does learning have to infer — and why is $\alpha$ not among them?**

$\pi$, every $\mu_k, \Sigma_k$ and every label $z_i$ — all **open**. $\alpha$ is a **dot**: fixed by us. Only the $x_i$ are observed:

$$p(\pi \mid \alpha) \prod_k p(\mu_k, \Sigma_k) \prod_i p(z_i \mid \pi) p(x_i \mid z_i, \mu, \Sigma)$$

A **mixture of Gaussians** — Lecture 07.

# Takeaway

A model diagram is a factorisation you can see: **shading** marks what the data fixes, **plates** mark what repeats independently *given* what sits outside, and **what sits outside** is what learning has to estimate.

# Inference and Parameter Learning

**Guiding Question:** We can now write the joint down. Can we actually condition on it — and where do its numbers come from?

# Three Questions You Can Ask a Model

| question | on the wet grass | with complete data |
|---|---|---|
| **infer** unobserved variables | $P(S \mid T, J)$ | cost set by the graph's shape |
| **learn the parameters** | the 8 table entries | counting |
| **learn the structure** | which arrows at all? | hardest — and only up to equivalence |

Speaker notes

Inference computes conditional distributions of unobserved variables given observed ones, with structure and tables known. Parameter learning fixes the graph and estimates the tables. Structure learning estimates the graph.

With complete data (every variable observed in every record) the difficulty order is often stated wrongly: parameter learning is the easiest, because the likelihood decomposes into independent counting problems; exact inference is NP-hard in general, though cheap on chains and trees; structure learning is the hardest, since it searches a super-exponential space of graphs and its result is determined only up to Markov equivalence. With incomplete data parameter learning requires inference as a subroutine (EM).

# Inference by Elimination

Explaining away needed $P(T{=}1, J{=}1)$. Sum the factorised joint — and **push each sum as far right as it goes**:

$$P(T{=}1, J{=}1) = \sum_R p(R)p(J{=}1 \mid R) \underbrace{\sum_S p(S)p(T{=}1 \mid R, S)}_{m(R)}$$

### Inner sum first

$m(1) = 0.1 \cdot 1 + 0.9 \cdot 1 = 1$

$m(0) = 0.1 \cdot 0.9 + 0.9 \cdot 0 = 0.09$

### Then the outer one

$0.2 \cdot 1 \cdot 1 + 0.8 \cdot 0.2 \cdot 0.09$

$= 0.2 + 0.0144 = 0.2144$ ✓

Speaker notes

The plain sum is $\sum_R \sum_S p(R)\,p(S)\,p(J{=}1 \mid R)\,p(T{=}1 \mid R,S)$. By distributivity, factors that do not mention $S$ can be moved out of the inner sum, leaving $m(R) = \sum_S p(S)\,p(T{=}1 \mid R,S)$, a table with one entry per value of $R$. $m(1) = 1$: given rain the lawn is wet whatever the sprinkler does. $m(0) = 0.09$: without rain only the sprinkler (on with 0.1, effective with 0.9) wets the lawn.

The result, $0.2144$, is the denominator of the explaining-away formula in Factorising the Joint. Summing out one variable at a time and storing the intermediate tables ("messages") is variable elimination. On four variables the saving is negligible; it grows with the number of variables, provided the intermediate tables stay small.

# Why Pushing Sums In Pays

A chain of $n$ binary variables $X_1 \to X_2 \to \cdots \to X_n$. To get $p(x_n)$:

$$2^{n-1}$$

terms if every other variable is summed at once — for $n = 30$, about 537 million

$$4(n-1)$$

operations if the sums are pushed in, one variable at a time — for $n = 30$, 116

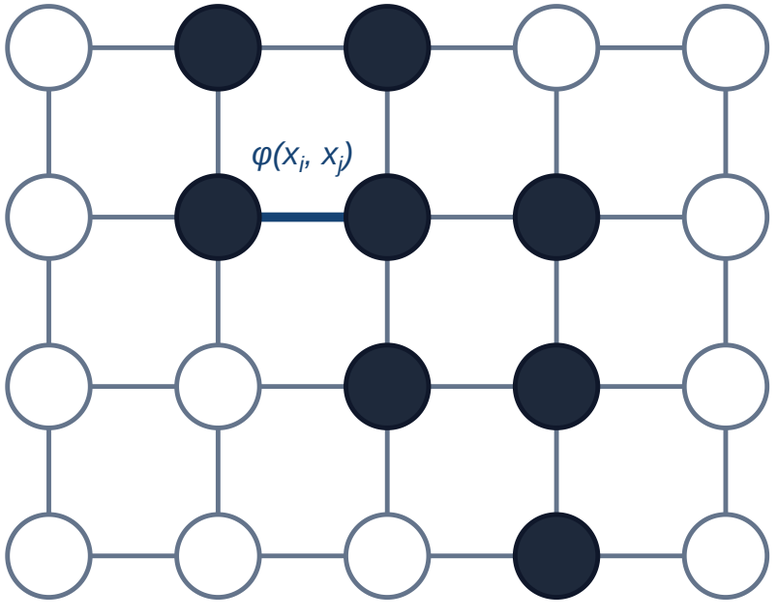
Eliminating $X_1$, then $X_2$, … passes a **two-entry table** along the chain. The cost is set by the **largest table** created on the way — by the graph's shape, not its size.

Speaker notes

Summing all of $X_1, \ldots, X_{n-1}$ at once enumerates $2^{n-1}$ configurations, each a product of $n$ factors: for $n = 30$, $2^{29} = 536{,}870{,}912$. Eliminating in order computes $m_1(x_2) = \sum_{x_1} p(x_1)\, p(x_2 \mid x_1)$, then $m_2(x_3) = \sum_{x_2} m_1(x_2)\, p(x_3 \mid x_2)$, and so on. Each step multiplies a two-entry message into a $2 \times 2$ table (four products) and sums out one variable, so $4(n-1)$ products in total: 116 for $n = 30$.

On a chain every elimination order keeps the intermediate tables at two variables; on general graphs a bad order creates large intermediate tables. Trees behave like chains. The general result: the cost of exact elimination is exponential in the treewidth of the graph (for the best order), roughly the number of variables in the largest table elimination has to build, and finding the best order is itself NP-hard.

# Undirected Models




each edge: a potential that prefers neighbours to agree

Pixels influence each other **symmetrically**. A **Markov network** puts a non-negative **potential** $\phi$ on each edge and normalises once, globally:

$$p(x) = \frac{1}{Z} \prod_{(i,j)} \phi(x_i, x_j), \qquad Z = \sum_x \prod_{(i,j)} \phi(x_i, x_j)$$

Two pixels, $\phi = 3$ if equal, $1$ if not: $Z = 3 + 1 + 1 + 3 = 8$, so $P(\text{equal}) = 6/8$.

Speaker notes

Potentials are non-negative functions, not probabilities: they need not sum to one, and a single potential has no probabilistic meaning on its own. Normalisation happens once, globally, through the partition function $Z$. Two pixels with $\phi = 3$ if equal and 1 if not: configurations (0,0), (0,1), (1,0), (1,1) have weights 3, 1, 1, 3, so $Z = 8$ and $P(\text{equal}) = 6/8 = 0.75$.

For an image, $Z$ sums over all $2^{\text{pixels}}$ binary images. Many Markov networks are written in log form, $p(x) \propto \exp(-E(x))$ with an energy $E$; the Ising model of statistical physics is this pixel model. Conditional independence in a Markov network is graph separation: $A \perp\!\!\!\perp B \mid C$ whenever every path from $A$ to $B$ passes through $C$ (the global Markov property; it holds for every distribution of this product form).

# Directed and Undirected

| | belief network | Markov network |
|---|---|---|
| edges | directed, acyclic | undirected |
| factors | conditional tables, normalised locally | potentials, one global $Z$ |
| natural for | causal or generative stories | symmetric interaction: pixels, neighbours |
| observing a common effect | creates dependence | — (no colliders) |

Inference runs on the undirected form: **moralisation** joins parents that share a child ($R$ – $S$ in the wet grass) and drops the arrow heads; elimination works on that graph.

Speaker notes

Moralisation ("marrying the parents") connects every pair of parents that share a child and drops all arrow heads. The resulting moral graph supports the same factorisation: each conditional table $p(x \mid \mathrm{pa}(x))$ is a potential over $x$ and its parents, which form a clique in the moral graph, and $Z = 1$. Exact inference algorithms (variable elimination, junction trees) operate on this undirected form.

Moralisation loses information: the moral graph of the wet grass joins $R$ and $S$, so it no longer shows that they are marginally independent. Hence independences are read off the DAG with d-separation, and computation runs on the moral graph. Neither model class contains the other as a set of independence structures: a collider ($a \perp\!\!\!\perp b$ but not given $c$) has no undirected graph with exactly those independences, and a four-cycle ($a \perp\!\!\!\perp c \mid b, d$ and $b \perp\!\!\!\perp d \mid a, c$) has no DAG with exactly those.

# Writable Is Not Queryable

A $100 \times 100$ black-and-white image as a pixel lattice:

**79,200**

numbers to **write it down**: 19,800 edges, 4 per potential

$$2^{100} \approx 10^{30}$$

entries in the table that elimination must build — a whole row of pixels at once

Exact inference in belief networks is **NP-hard** in general, and so is approximate inference with a guaranteed error bound. What remains: exploit the shape where it is kind (chains, trees), and otherwise **sample** or **approximate** — trading the guarantee for an answer.

Speaker notes

A $100 \times 100$ lattice has $100 \cdot 99 = 9{,}900$ horizontal and as many vertical edges, 19,800 in total; with a $2 \times 2$ potential each, 79,200 numbers. Eliminating row by row creates intermediate tables over an entire row of 100 pixels, $2^{100} \approx 1.3 \cdot 10^{30}$ entries. This is not an artefact of a bad order: the treewidth of an $n \times n$ grid is $n$, so every elimination order produces a table over at least about 100 variables. $Z$ itself is a sum over $2^{10{,}000}$ images.

Hardness results: exact inference in belief networks is NP-hard (Cooper 1990); approximating posterior probabilities with a guaranteed error bound is NP-hard as well (Dagum and Luby 1993). In practice such models are handled with sampling (Markov chain Monte Carlo) or variational approximations, or with exact algorithms where the graph structure allows (chains, trees, low treewidth). EM (Lecture 07) alternates between inference over latent variables and refitting the parameters.

# Learning the Parameters Is Counting

Fifty observed days. Estimate Jack's table $p(J{=}1 \mid R)$:

| | days | Jack's lawn wet | MLE (peak) | $+1$ pseudo-counts (mean) |
|---|---|---|---|---|
| $R = 0$ | 40 | 8 | $8/40 = 0.20$ | $9/42 \approx 0.21$ |
| $R = 1$ | 10 | 10 | $10/10 = 1.0$ | $11/12 \approx 0.92$ |

> Counting **is** maximum likelihood (Lecture 03); pseudo-counts **are** the Beta posterior mean (Lecture 03) — Laplace smoothing (Lecture 03). "Rain always wets the grass" was an MLE on ten days.

**Hidden** variables leave no counts: infer them, re-count, repeat — EM, Lecture 07.

Speaker notes

With complete data the log-likelihood of a belief network is a sum over nodes and parent configurations of terms that involve only one row of one table. Each row is therefore maximised separately, and the maximiser is the relative frequency: $\hat{p}(J{=}1 \mid R{=}r) = n(J{=}1, R{=}r)/n(R{=}r)$. Naive Bayes is the special case with the star graph.

With a $\mathrm{Beta}(1,1)$ prior on each row the posterior mean is $(n_1+1)/(n+2)$: $9/42 = 0.214$ and $11/12 = 0.917$. The 1.0 in the table of Factorising the Joint is a maximum-likelihood estimate from ten days without a single counterexample, the same situation as three heads in three flips in Lecture 03. A parent configuration that never occurs leaves its row undetermined (0/0); the pseudo-counts fill it with the prior mean 1/2. With hidden variables the counts are unavailable; EM replaces them by expected counts under the current posterior.

# Learning the Structure

### Score and search

Try graphs, score each by how well it fits the data *per parameter*, keep the best. The number of DAGs grows super-exponentially.

### Test independences

Run conditional-independence tests and keep the graphs consistent with them — d-separation, read backwards.

Either way, data determines a graph only **up to Markov equivalence**: arrows between equivalent graphs are not in the data. A learned structure is not a causal discovery.

Speaker notes

Score-based methods search over graphs and maximise a score that trades fit against complexity, such as BIC (log-likelihood minus $\frac{1}{2}\log N$ per free parameter) or a Bayesian marginal likelihood. The number of DAGs on $n$ labelled nodes is 1, 3, 25, 543 for $n = 1, \ldots, 4$ and about $4 \cdot 10^{18}$ for $n = 10$, so search is heuristic (greedy, with local moves). Constraint-based methods (e.g. the PC algorithm) run conditional independence tests and keep graphs whose d-separations match them; errors in individual tests propagate.

Both recover at best the Markov equivalence class, and only under assumptions: enough data, no unobserved common causes, and a distribution faithful to some DAG. A learned structure is therefore not a causal discovery; causal conclusions require interventional data or explicitly justified assumptions.

# Mini-Exercise

Jack's lawn was wet on all **4** rainy days in a small sample.

**What does counting give for $p(J{=}1 \mid R{=}1)$ — and what does one pseudo-count on each side give?**

Speaker notes

MLE: $4/4 = 1$. With one pseudo-count on each side, i.e. a $\mathrm{Beta}(1,1)$ prior, the posterior is $\mathrm{Beta}(5,1)$ with mean $5/6 \approx 0.833$; its mode is still 1, the MLE. The three readings connect Lecture 03 (Beta posterior mean), Lecture 03 (the peak of a small sample can lie) and Lecture 03 (Laplace smoothing).

The prior's influence shrinks with $n$: with 400 wet days out of 400 the smoothed estimate is $401/402 \approx 0.9975$, against the MLE 1. The difference of $1/(n+2)$ vanishes as $n$ grows, but the MLE never leaves the boundary value 1 however many confirming days are counted.

# Mini-Exercise

Jack's lawn was wet on all **4** rainy days in a small sample.

**What does counting give for $p(J{=}1 \mid R{=}1)$ — and what does one pseudo-count on each side give?**

**Counting:** $4/4 = 1$ — "rain always wets the lawn", from four days. **Smoothed:** $5/6 \approx 0.83$, the posterior mean under $\mathrm{Beta}(1,1)$. Same data, two readings: the peak and the mean.

# Takeaway

A graphical model makes the joint **writable**; its **shape** decides whether it is **queryable** — elimination is cheap on chains and trees and hopeless on a lattice. With complete data the parameters are counts: the MLE, or with pseudo-counts the Beta posterior mean. The structure is learnable only up to equivalence.

# Appendix

# Uncertain and Unreliable Evidence

## Uncertain (soft) evidence

We are not sure of the observation itself: a distribution $\tilde{x}$ over its states. Sum over them:

$$p(y \mid \tilde{x}) = \sum_{x} p(y \mid x) p(x \mid \tilde{x})$$

*We look at the lawn from the window without touching it:* $p(T{=}1 \mid \tilde{T}) = 0.8$.

## Unreliable evidence

We keep the report but **replace** its table by our own judgement of how reliable it is:

$$p(x \mid \mathrm{pa}(x)) \to \tilde{p}(x \mid \mathrm{pa}(x))$$

*Jack says his lawn is wet, but he did not check.*

Both are drawn with a **dashed** edge.

Speaker notes

Uncertain (soft, or virtual) evidence keeps the model unchanged and replaces a hard observation $x = x^*$ by a distribution over the states of $x$; the query averages over those states. With $p(T{=}1 \mid \tilde{T}) = 0.8$ and the wet-grass tables, $p(S{=}1 \mid \tilde{T}) = 0.8 \cdot p(S{=}1 \mid T{=}1) + 0.2 \cdot p(S{=}1 \mid T{=}0) = 0.8 \cdot 0.338 + 0.2 \cdot 0.011 \approx 0.273$.

Unreliable evidence keeps the reported value but changes the model of how the report arises: the table of the reported variable is replaced by one that expresses how far the reporter is trusted. The two are frequently confused. A rumour is not an uncertain observation of the lawn; it is a certain observation of a report whose relation to the lawn is unreliable, and treating it as noise on the lawn itself models the wrong variable.

This follows the treatment of soft and unreliable evidence in Barber, *Bayesian Reasoning and Machine Learning*, Chapter 3.